



Chapter 2

Encapsulation

Concepts and Principles: Abstraction, assertion, class, design by contract, encapsulation, immutability, information hiding, input validation, interface, object diagram, record, scope.

Patterns and Antipatterns: INAPPROPRIATE INTIMACY[†], PRIMITIVE OBSESSION[†].

An essential technique in software design is to decompose a system into distinct, manageable abstractions. However, there is little value in decomposing a piece of software into several parts if each part depends on all the other parts in a tangled mess of interactions. For a decomposition to be useful, the resulting abstractions have to be well isolated from each other. For good design, an idea that should be inseparable from that of software abstraction is *encapsulation*.

Design Context

We start our exploration of software design by considering how to effectively represent a deck of playing cards in code. This representation would be necessary for most computer card games, for example the Solitaire game used as a sample application. In the card deck used as a running example, there are 52 distinct cards and any given card can be completely defined by its *suit* (Hearts ♥, Spades ♠, Diamonds ♦, Clubs ♣) and its *rank* (Ace, 2, 3, ..., 10, Jack, Queen, King). A software structure to represent a deck of cards should therefore be able to represent any sequence of any number of distinct cards between 0 and 52. Two main operations required of a deck of cards are to *shuffle* it and to *draw* cards from it. Shuffling randomly reorders the cards in the deck. In the domain of card games, *drawing* a card means to remove it from the deck (typically from the top). This operation is not to be confused with the action of depicting the card on a user interface component.

2.1 Encapsulation and Information Hiding

The idea of *encapsulation* is to enclose something as if it were in a capsule. For example, we can think of a nut, which is encapsulated in its shell. The shell, or capsule, serves as protection. In software design we encapsulate both data and computation to limit the number of contact points between different parts of the code. Encapsulation has several benefits: it makes it easier to understand a piece of code in isolation, it makes the use of the isolated part by the rest of the code less error-prone, and it makes it easier to change one part of the code without breaking anything. In software design, the equivalent of a shell is the general concept of an *interface*.

Encapsulation helps to apply the principle of *information hiding*, which has been around since the early 1970s. Following the principle of information hiding, encapsulated structures should only reveal the minimum amount of information that is necessary to use them, and hide the rest. A typical example of information hiding is an implementation of a stack abstract data type (ADT) whose interface only provides *push* and *pop* operations. This minimal interface allows *client code* to make use of the stack structure, but decisions on how to store elements in the stack remain hidden from the code that uses the stack. I use the term *client code* to refer to any code that uses a code element that is not part of the definition of this element. The term *code* in *client code* is especially important, because here *client* does not refer to a customer or user of a software project. Which part of the code qualifies as *client code* will depend on the situation at hand. In many cases, the details of the client code will not really matter in the discussion of the design ideas.

Although information hiding is a principle of software design that is very general, there exist specific techniques that we can use to help ensure our code is well-encapsulated and respects the principle. The rest of this chapter presents some of these techniques.

2.2 Encoding Abstractions as Types

As our first design task, we define the abstractions that are necessary to represent a deck of cards. An *abstraction* is a conceptual building block for a software system. Examples of common abstractions in computing include data structures (for example, stack, list) and operations (sorting, iterating). However, abstractions can also refer to ideas in the problem domain, such as *playing card*. With the term *defining* an abstraction, I mean deciding what the abstraction represents, and what it will look like in terms of source code. In the case of a deck of cards, the first part of the process is straightforward, because the concepts we need to represent in code (a playing card, a deck of cards) are well-defined in the real world. This will not always be the case.

Essentially, a deck of cards is an ordered collection of playing cards. We could use any standard data structure to represent this collection (an array, a list, etc.). However, what would such a collection hold? What is a card? In the code, we can

represent a playing card in many different ways. For example, we could use an integer between 0 and 51 where the value represents a certain card according to a convention. For example, Clubs could have numbers 0–12 in increasing rank, Hearts 13–25, etc.:

```
int card = 13;           // 13 = The Ace of Hearts
int suit = card / 13;    // 1 = Hearts
int rank = card % 13;    // 0 = Ace
```

This approach would also require us to have similar conventions to represent suits and ranks, as illustrated on the second and third lines.¹ To avoid having to continually divide and multiply numbers that represent cards to switch between suits, we could also represent a card as a pair of values, the first one encoding the suit, and the second one encoding the rank (or vice-versa):

```
int[] card = {1,0}; // The Ace of Hearts
```

While we are at it, we could even decide to represent a card using a combination of six Boolean values. Although extremely inconvenient, this design decision is technically possible to implement: it is an example of a decision that is possible, but not acceptable (see Section 1.1). As it turns out, all three options above have major drawbacks.

First, *the representation of a card does not map to the corresponding domain concept*. To facilitate code understanding and help avoid programming errors, the representation of values should ideally be tied to the concept they represent. For example, the general type `int` maps to the concept of an integer (a type of number), not that of a playing card. We could define a variable of type `int` intended to store a playing card, and unwittingly put a value that represents a different entity in it (e.g., the number of cards in the deck). This will not be noticed as an error by the compiler, yet it is likely to lead to intense confusion when executing the code.

Second, *the representation of a card is coupled with its implementation*. If our design decision is that cards should be represented as integers, any location in the code that must store a value that represents a card will refer to an integer. Changing this encoding to something else (for example, the two-element array discussed above) will require discovering and changing every single location where an `int` variable is used to store a card, and all the code that works with cards as integers.

Third, *it is easy to corrupt a variable that stores a value that represents a card*. In Java a variable of type `int` can take 2^{32} distinct values. To represent a playing card we only need a tiny subset of these (52 values). Consequently, the overwhelming majority of values we can store in an `int` variable intended to represent a playing card ($2^{32} - 52$) does not represent any valid information. This opens the door to errors. The problem would have been even worse had we decided to use a two-element array of type `int`, which supports $2^{64} + 1$ values.²

We can do better. It is generally a bad idea to try to shoehorn domain concepts into basic types such as `int`, `String`, and so on. Ideally, these types should only

¹ The modulo operator (%) returns the remainder of the integer division.

² The additional value comes from the fact that array-typed variables can also be null.

be used to hold values that are proper values of the type. For instance, the `int` type should only be used to hold actual integers (and perhaps very similar concepts, such as currency). Similarly, `Strings` should be used only to hold sequences of characters meant to represent text or text-like information, as opposed to being some encoding of some other concept (for example, "AceOfClubs"). The tendency to use primitive types to represent other abstractions is a common antipattern called PRIMITIVE OBSESSION[†].

To apply the principle of information hiding, we instead organize our code to *hide* the decision of how exactly we represent a card. We hide this decision behind an interface specifically tied with the concept of a card. In programming languages with a strong support for types, such as Java, this is typically accomplished through the use of types. In our case, to properly represent a card in code, we define our own type `Card` as a Java class:

```
class Card {}
```

As we will see below, the use of a specific type to represent a card will allow us to hide the decision of how we represent a card internally. However, although we now have a class `Card`, we still need to decide how to represent a card within the class. All options are back on the table. We could do simply:

```
class Card {
    int aCard; // 0-51 encodes the card
}
```

This class defines a single instance variable `aCard` of type `int`. The name of the instance variable includes the prefix `a` as part of a coding convention detailed in Appendix B. Client code can refer to this variable through *dereference* (see Section A.2), for example:³

```
Card card = new Card();
card.aCard = 28;
```

Although using a class links the value to the domain concept of a card, the other problems persist. First, it is still possible to corrupt the representation of a card. Second, the decision to represent this value as an `int` is not exactly hidden, given that client code would be accessing the variable directly through a dereference of the instance variable. Let us then tackle the issue of representing the card internally. The next section handles the issue of hiding this decision.

Two key observations can help us arrive at a better way to encode a card. First, the value of a playing card is completely and exactly defined in terms of two sub-concepts: its suit (e.g., Clubs) and its rank (e.g., Ace). So, we can take the process of defining types one step further, and define abstractions for ranks and suits. Following the same reasoning as above, primitive values are not a good match for encoding abstractions of a rank and a suit, so we use a dedicated type for each. However, here, the second important observation comes into play: the rank of a playing card can

³ Technically, this code only compiles if placed in a method declared in a class that is in the same package as class `Card`. This detail is not important here. Section 2.3 explains where class members can be accessed in the code.

only be one of 13 distinct values, which are known in advance and can be enumerated. In the case of suits, the number of values is even smaller (four). The best tool at our disposal to encode such abstractions is the *enumerated type*:

```
enum Suit {
    CLUBS, DIAMONDS, SPADES, HEARTS
}
```

In Java, enumerated types are a special kind of class declaration. The identifiers listed in the declaration of the enumerated type are globally available constants (see Section A.3 in the appendix). These constants store a reference to an object of the class that corresponds to the enumerated value. For example,

```
Suit suit1 = Suit.CLUBS;
Suit suit2 = Suit.CLUBS;
boolean same = suit1 == suit2; // same == true
```

Enumerated types are a great fit here. They meet all our design requirements, because variables of type `Suit` and `Rank` are directly tied to their corresponding concept of rank and suit, and variables of these types can only take values that are meaningful for the type.⁴ Enumerated types are a simple yet effective feature for realizing robust designs. They help avoid PRIMITIVE OBSESSION† and generally make the code clearer and less error-prone.

The code below completes our definition of class `Card` as a combination of a rank and a suit value. It assumes that each enumerated type is defined in its own file.

```
enum Suit {
    CLUBS, DIAMONDS, SPADES, HEARTS
}

enum Rank {
    ACE, TWO, ..., QUEEN, KING
}

class Card {
    Suit aSuit;
    Rank aRank;
}
```

Now that we have a reasonable type to represent a playing card in the code, we return to the issue of representing a deck of cards, and again follow the same reasoning. Because a deck is just a collection of cards, we could represent a deck of cards as a `List of Cards`:

```
List<Card> deck = new ArrayList<>();
```

However, the disadvantages of this approach are the same as the disadvantages of representing a playing card as an `int` value:

- A list of cards is not strongly tied to the concept of a deck. It could represent any list of cards, e.g., the cards in one of the piles created while playing Solitaire, the cards discarded as part of the game, etc.

⁴ With the unfortunate exception of `null`. See Section 4.4.

- Using a list of cards ties the representation of a deck in the program with its implementation. If we decide later to replace the list by, say, an array, we would have to change all the corresponding source code locations.⁵
- The structure can easily be corrupted: a simple deck of cards can hold a maximum of 52 cards, without duplicates. A list allows one to put any number of cards in the structure, including duplicates.

A better way to approach the representation of a deck of cards in our code is to also define a proper type for it:

```
class Deck {
    List<Card> aCards = new ArrayList<>();
}
```

Although it may seem redundant to define a new class to hold just one instance of an `ArrayList`, this decision helps avoid many of the problems discussed above. The new type `Deck` specializes the list and ties it directly to its corresponding domain concept. It also becomes possible to hide the decision of how the cards are stored. The remainder of this chapter presents the details of how to achieve this hiding in practice.

Code Exploration: Solitaire · Card

A complete version of the `Card` class.

This boxed paragraph, called a Code Exploration, is the first discussion of design decisions based on the code of the sample applications. The left part of the title is the name of the sample application, and the right part is the name of the class discussed. See Appendix C for instructions on how to access the relevant code.

This chapter has used the creation of a deck of cards as a running example, so it is worth pointing out class `Card` in the Solitaire project. Some of the code in the class implements more advanced features that I will return to in Chapter 4, including the static members and the private constructor. Ignoring these, however, the basic structure of the class is identical to the one discussed in this section: two fields of enumerated types to represent the card's rank and suit, respectively, as well as two accessor methods to obtain these values from a `Card` object.

⁵ Disturbingly, replacing the list by a `Stack` on the right-hand side of the assignment in the listing would actually work because in Java `Stack` is a subtype of `List`. In Chapter 7, I explain why this is disturbing.

2.3 Scopes and Accessibility

Encoding abstractions as types is only the first step in the process of encapsulation. Once we have types we feel are good abstractions for our design, we need to ensure that these types are effective in hiding information from client code. At this point we have determined that four types are necessary to represent a deck of cards in code: `Deck`, `Card`, `Rank`, and `Suit`. Each of these types defines a set of possible values and a set of operations on these values. We now turn to the problem of specifying the values these types can take and the operations on these types so as to achieve good encapsulation of both the values and computation.

In Java and most other object-oriented languages, an *object* is a mechanism to group variables together and access their values through the process of dereferencing (see Section A.2 in the appendix). Without encapsulation, any variable that forms part of an object can be accessed indiscriminately. For example, given the following code:

```
class Deck {
    public List<Card> aCards = new ArrayList<>();
}

class Card {
    public Rank aRank = null;
    public Suit aSuit = null;
}
```

we could use our objects as follows:⁶

```
Deck deck = new Deck();
deck.aCards.add(new Card());
deck.aCards.add(new Card());
deck.aCards.get(1).aRank = deck.aCards.get(0).aRank;
System.out.println(deck.aCards.get(0).aSuit.toString());
```

Because of the complete lack of encapsulation, we can make unprincipled use of the internal implementation of our types. Without major effort, this kind of code invariably leads to bugs, because the number of ways to misuse the structures greatly exceeds the number of ways to use them properly. For example, although it may not be immediately apparent, the code above, when executed, raises a `NullPointerException`. With good encapsulation, it should be near-impossible to misuse one of our types.

The idea of encapsulation is to hide the internal implementation of an abstraction behind an interface that tightly controls how an abstraction can be used. Designing good abstractions and good interfaces for these abstractions are tandem tasks that underlie most of software design. Designing effective interfaces can be tricky and requires a combination of different mechanisms and techniques. We start with one of the simplest, *access modifiers*. Access modifiers are Java keywords that control what parts of the code can *access* certain program elements (e.g., classes, fields,

⁶ This code is not defined in any method because its exact location does not matter. For example, the code could be placed in a `main` method.

methods). Controlling access to fields is a concept similar to that of visibility and *scope* for local variables. In most programming languages, a *scope* is a lexical region that acts as a boundary for variables. In Java, scopes are defined using curly braces. The following code fragment:

```
public static void main(String[] args) {  
    { int a = 0; }  
    { int b = a; }  
}
```

has a compilation error because, in the second assignment, the reference to `a` cannot be resolved because, according to Java scoping rules, it is not visible in the second scope. In this tiny example, the scoping restriction may look like a limitation. However, scopes are a powerful feature. To understand what happens in the second statement, we only need to track down references to variables that are in scope (as opposed to every code location). We can do the same with classes.

In Java, it is possible to control the visibility of classes and class members (and in particular fields) through the use of access modifiers. In this chapter, I only focus on the distinction between the `public` and `private` access modifiers.⁷ Members marked `public` are visible anywhere in the code. In the example above, because the field `aCards` of class `Deck` is `public`, the variable `aCards` of any object of type `Deck` is accessible from any code that has a reference to an object of that class. In contrast, members marked `private` are only visible within the scope of the class, namely, between the opening curly brace for the declaration of the class body and the closing curly brace of the class declaration.

A guideline for achieving good encapsulation is to use the narrowest possible scope for class members. Thus, instance variables should almost always be `private`. Also, `public` methods should reveal as little as possible about implementation decisions meant to be encapsulated. A revised design for class `Card` that respects this guideline is as follows:

```
public class Card {  
    private Rank aRank;  
    private Suit aSuit;  
  
    public Card(Rank pRank, Suit pSuit) {  
        aRank = pRank;  
        aSuit = pSuit;  
    }  
  
    public Rank rank() {  
        return aRank; }  
  
    public Suit suit() {  
        return aSuit; }  
}
```

⁷ The other two are `protected` and *default* (absence of a modifier). Members with default visibility are accessible by code in classes declared in the same package. I cover the `protected` modifier in Chapter 7.

This class properly encapsulates the representation of a playing card because client code cannot interact in any way with the internal representation of a card. In fact, with this design it is possible to change the representation of the card to use a single field of type `int`, or an enumerated type (say, `PlayingCard`), without requiring *any* change to the client code.

As a mechanism for software design, access modifiers serve a dual purpose. First, they express the *intent* of the developer about where certain structures are meant to be used. Second, they support the automatic enforcement of the stated intent through compilation. In an ideal design, the intent of the developer should be clear. Access modifiers also help provide us with our first working definition of a type's interface in Java. In general, an interface to a class consists of the methods of that class that are accessible to another class. For now, we will keep things simple and consider that the interface to a class is the set of its public methods. This is only a starting point, and we will be refining this definition of interface as we go along, and in particular in Section 3.1. For now, what is important is that the public methods of a type (a class) represent what client code can do with objects of the type, and the design of all other (non-public) fields and methods remains hidden from that client.

So far, our `Card` type is a simple combination of two fields: `aRank` and `aSuit`. The value of each field is set through the constructor and can be accessed through a getter method that bears the name of the field (excluding the prefix of the naming convention). Since version 14, Java provides a feature called *records* to simplify the declaration of such basic types. The record declaration equivalent to the class above is as follows:

```
public record Card (Rank rank, Suit suit) {}
```

This declaration defines a type with two fields, `rank` and `suit`. From this declaration, a constructor is generated that directly assigns arguments to the fields indicated in the type declaration. In addition, one getter method per field is generated, which receives the same name as the field.⁸ With the record declaration above, it is now possible to write client code such as:

```
Card card = new Card(Rank.ACE, Suit.CLUBS);
Rank rank = card.rank();
Suit suit = card.suit();
```

Records have other interesting features, including that they can declare additional constructors and methods. I introduce these features as they become relevant.

Code Exploration: JetUML · Dimension

Avoiding PRIMITIVE OBSESSION† with a small abstraction.

At first glance, the `Dimension` type, defined as a record, looks exceedingly simple: a pair of integer values, one to represent a width, one to represent a height. Why bother, since these values can be accessed individually (using the

⁸ For this reason, I do not follow the naming convention prefixing field names with `a` for records, as this would propagate the prefix to the name of the getters.

generated getter methods `width()` and `height()`? Would it not be simpler to just use pairs of integers? Is this not excessive effort to avoid a case of PRIMITIVE OBSESSION†? The answers to these questions are not to be found in the definition of the record itself, but rather by looking at all the places in the code where `Dimension` is used. The rationale for encoding the concept of a *dimension* explicitly is threefold: to be able to return both related values as one object and to prevent errors caused by flipping the width and the height. Of course, these have to be provided in the right order in the constructor call, but once a `Dimension` object is created, the risk of flipping the two values is eliminated. This definition of type `Dimension` also illustrates how records can define additional fields and methods.

2.4 Object Diagrams

An object diagram is a type of UML diagram (see Section 1.3) that represents objects and how they refer to each other. Whenever a `new` expression is evaluated, an object of a class is created and a reference to this object is returned and can be passed around. It can often be useful to visually represent the resulting graph of objects and their inter-dependent references.

I introduce a slight enhancement to official UML object diagrams, so as to provide a representation of an object's fields and values that resembles the kind of data-structure diagrams often used in introductory computer science classes. In an object diagram, a rectangle represents an object, with its name and type indicated as `name:type`. Both name and type information are optional, but in general it is useful to have at least one of the two. In UML diagrams in general, the name of objects (as opposed to classes) are underlined. Objects can contain *fields*, which are just like fields in a Java program. Fields can contain a value of a primitive type or a value of a reference type (see Section A.1 in the appendix). When the value is a reference type, it is represented as a directed arrow to the object being referenced.

Let us consider the diagram of Figure 2.1. This diagram models an instance of the `Deck` class named `deck`. It would have been fine to omit this name and simply indicate `:Deck` in the rectangle, as in the case of `ArrayList<Card>`, which is anonymous. This `deck` has a field `aCards` whose current value is a reference to an `ArrayList<Card>` object. The `ArrayList<Card>` object's `elementData` field references two `Card` instances. Here, because `ArrayList` is a library type, it is necessary to have knowledge of the source code of the library to accurately model objects of this class. However, for a design sketch, using the actual name is not critical. To model internal properties of library types without looking up all their implementation details, it is often sufficient simply to make up evocative names. For example, the diagram would be just as informative if the field had been named `data` or `elements`.

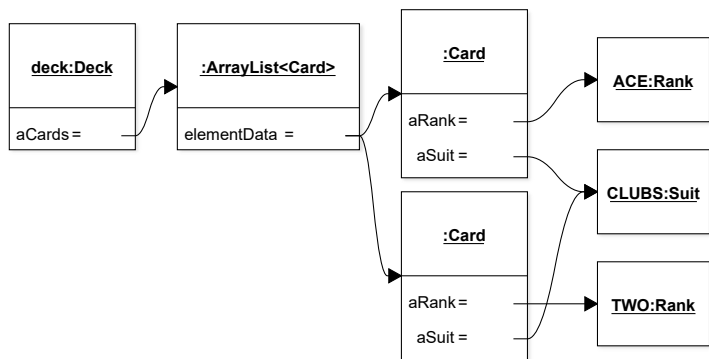


Fig. 2.1 Object diagram showing a detailed model of the object graph for a deck of cards

Through modeling, we can skip over some details. In reality, in an instance of `ArrayList` the `elementData` field refers to an array of `Object`-typed cells that contain the actual data. This information is not useful here, and we link directly to the contained data. It is also worth noting how the list refers to two cards, and not three or four or 52. Another important point about object diagrams is that they represent a *snapshot* in the execution of a program. Here it was at the point where the list had two cards. For the purpose of communicating design information, including only two cards is sufficient to illustrate that a deck is a list of cards, so it would not be worth it to depict a snapshot of the program when the deck contains more cards. The two `Card` instances, however, are modeled in full detail. The values of enumerated types are distinguished by name, as they should be, and the enumerated value `Suit.CLUBS` is *shared* between two cards.

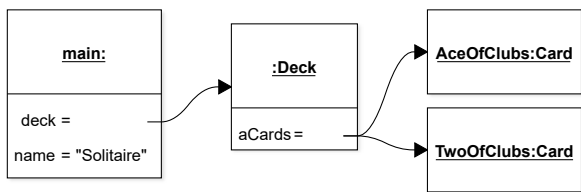


Fig. 2.2 Object diagram showing a simplified model of the object graph for a deck of cards

The second example diagram (Figure 2.2) illustrates some of the additional modeling simplifications we can do, when appropriate. First, I added an untyped object named `main`. This “object” is actually a trick for representing a method body. Object diagrams do not have an explicit notation to represent code statements that form the body of a method declaration. However, this can be achieved through untyped objects by observing that, from the point of view of the diagram, an object and a method body are both collections of variables (instance variables in the first case,

local variables in the second). A second difference is that the `Deck` object is now anonymous, and the name `deck` is used to represent the *variable* in which a reference to an object (any object) is stored, as opposed to a specific object. Third, the `main` method contains a `name` variable that stores a string. In Java, strings are technically instances of the reference type `String`. To be strictly accurate, we should represent a string value as a reference to an instance of class `String` that has a reference to an array of `char` values, each with one letter. That level of detail would be both superfluous and annoying, so we just show the string literal. A fourth important difference is that the `ArrayList` has been abstracted away. In this diagram, we see that a `deck` somehow keeps track of a number of cards in a field `aCards`, but how these are stored internally is not represented. The cards could be in an array, a list, whatever. Although in some cases (such as in the next section) the details may be important, it is often the case that details of internal data structures are superfluous. Finally, the value of the `Card` instances are represented artificially by using an evocative name for the objects, instead of modeling the field values. This does not mean that these `Card` instances do not have the `aRank` and `aSuit` fields, it just means this detail has been elided from the diagram.

2.5 Escaping References

The use of the visibility restrictions for fields using the `private` keyword provides a basic level of encapsulation, but it by no means ensures an iron-clad protection of internal structures. We explore this problem by returning to the issue of storing an aggregation of `Card` objects within an instance of a `Deck` object. Let us assume we decided to implement a `Deck` as a list of cards using Java's `ArrayList` type.⁹

```
public class Deck {

    private List<Card> aCards = new ArrayList<>();

    public Deck() {
        /* Add all 52 cards to the deck */
        /* Shuffle the cards */
    }

    public Card draw() {
        return aCards.removeLast();
    }
}
```

So far, the only way to use an instance of `Deck` from code outside the class is to *draw* a card from the deck: there are no other members (methods or fields) that could be referenced outside the class. The class is thus well encapsulated, but also

⁹ It may appear that `Stack` could be a better choice, but I prefer to avoid this type because its implementation is victim of a design flaw discussed in Chapter 7.

limited it the services it can offer. Let us assume the client code needs to inspect the content of the deck. We could simply add a *getter* method to the class:

```
public class Deck {  
  
    private List<Card> aCards = new ArrayList<>();  
  
    public List<Card> cards() {  
        return aCards;  
    }  
}
```

Unfortunately, this solution solves the problem of providing access to the content of the deck at a great cost: it allows a reference to the private internal list of cards to *escape* the scope of the class, thus granting access to internal elements of the class from outside the class. For example:

```
Deck deck = new Deck();  
List<Card> cards = deck.cards();  
cards.add(new Card(Rank.ACE, Suit.HEARTS));
```

Here, the reference to the list of cards held within an instance of `Deck` *escaped* into the scope of the client code, which can then use it to mess things up, for example by adding an additional Ace of Hearts.

Clearly, declaring fields private is insufficient to ensure good encapsulation. If a class is well encapsulated, it should not be possible to change the data stored by an object without going through one of its methods. In turn, to achieve this encapsulation quality, it is also necessary to prevent references to internal structures to escape the scope of the class. There are three main ways in which a reference to a private structure can escape the scope of its class: returning a reference to an internal object, storing an external reference internally, or leaking references through shared structures.¹⁰

Returning a Reference to an Internal Object

This problem is demonstrated above through the use of the getter method. It is not a good idea to automatically supply getters and setters for each field because, as in this case, it may result in a degradation of encapsulation. Figure 2.3 shows the effect of this escape.

Although an object is a collection of variables, in the context of design, these variables correspond to an abstraction. Having a class that is mostly accessed through getters and setters points to a design weakness, because the abstraction the object represents is not effective. This problem is also known as the `INAPPROPRIATE INTIMACY`† antipattern, because its symptom is that classes “spend too much time delving in each others’ private parts” [4]. To the extent possible, objects should interact with each other using methods that involve abstractions above individual instance vari-

¹⁰ A fourth, more indirect, way is to use metaprogramming. See Section 5.4.

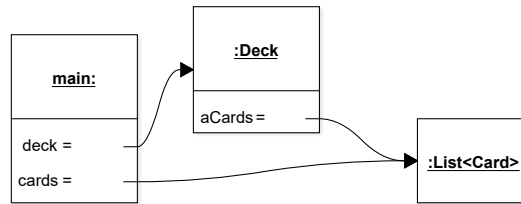


Fig. 2.3 Effect of leaking a reference outside the class scope

ables. In the case of the `Deck` class, this means prohibiting access to the internal list of cards, which constitutes a “private part”.

Storing an External Reference Internally

The problem with returning a reference to an internal object is that this reference becomes shared by the client code. A similar problem is to introduce this sharing by using a reference to an external object to initialize the internal state of another object. For example, if we have a setter method for the content of the deck:

```
public class Deck {

    private List<Card> aCards = new ArrayList<>();

    public void setCards(List<Card> pCards) {
        aCards = pCards;
    }
}
```

the reference will already be escaped as soon as it is assigned to the field:

```
List<Card> cards = new ArrayList<>();
Deck deck = new Deck();
deck.setCards(cards);
cards.add(new Card(Rank.ACE, Suit.HEARTS));
```

Here, we can corrupt the state of the deck from the scope of the client code, for example by adding an Ace of Hearts. From an object graph perspective, the outcome of this code is similar to the one caused by leaking a reference through a getter method, as illustrated in Figure 2.3.

A similar version of this problem is to set the content of the deck from a constructor, as opposed to a setter method:

```
public class Deck {
    private List<Card> aCards;

    public Deck(List<Card> pCards) {
        aCards = pCards;
    }
}
```

Although the leak uses a different type of programming language element (constructor vs. setter method), the result is identical.

Leaking References Through Shared Structures

The issue of escaping references is complex because references can escape through any number of shared structures, which may not always be obvious. Although contrived, the following example shows how this could come about:

```
public class Deck {
    private List<Card> aCards = new ArrayList<>();

    public void collect(List<List<Card>> pAllCards) {
        pAllCards.add(aCards);
    }
}
```

with the corresponding client code:

```
List<List<Card>> allCards = new ArrayList<>();
Deck deck = new Deck();
deck.collect(allCards);
List<Card> cards = allCards.get(0);
cards.add(new Card(Rank.ACE, Suit.HEARTS));
```

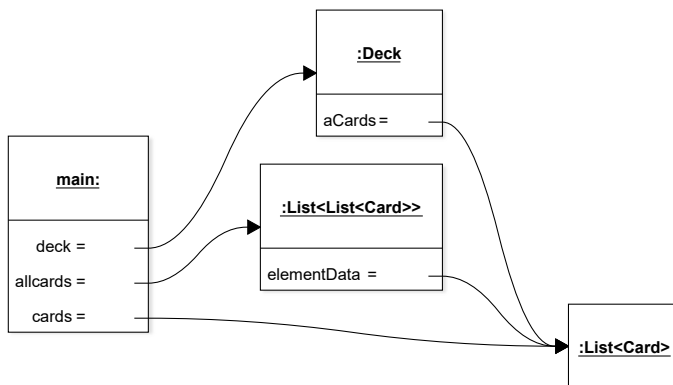


Fig. 2.4 Effect of leaking a reference through a shared structure

Figure 2.4 illustrates the result. Unfortunately, automatically detecting escaping references is a difficult program analysis problem, and there currently does not exist any production tool that can accomplish it for Java. Preventing the escape of references from the class scope is currently a manual process that relies on rigorous programming and code inspection practices. Section 2.7 introduces techniques for exposing some carefully selected information encapsulated by an object, without leaking references to internal structures.

2.6 Immutability

One of the major design insights of this chapter is that to ensure good encapsulation, it should not be possible to modify the internal state of an object without going through its methods. Section 2.5 discussed the issue of escaping references, and how they threaten encapsulation. There is, however, one situation where leaking a reference to an internal object is harmless: when the object is immutable (i.e., impossible to change). Let us consider the following code:

```
class Person {
    private String aName;

    public Person(String pName) {
        aName = pName;
    }

    public String name() {
        return aName;
    }
}

public class Client {
    public static void main(String[] args) {
        Person person = new Person("Anonymous");
        String name = person.name();
    }
}
```

The implementation of class `Person` clearly violates the advice given in Section 2.5 (of not returning references held in private fields), given that `Person.name()` returns a reference to the value of an instance variable. We can also represent this situation with an object diagram (Figure 2.5):

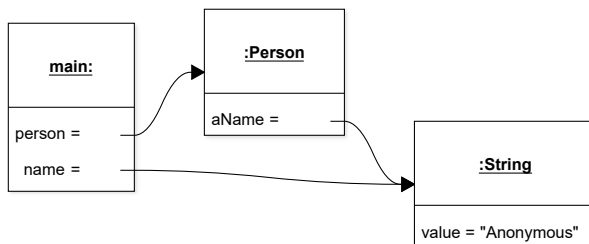


Fig. 2.5 Illustration of a shared reference to a `String` instance

However, a crucial notice in the reference documentation of the `String` library type changes things considerably:

Strings are constant; their values cannot be changed after they are created. [...] Because String objects are immutable they can be shared.

Because it is not possible to change the data encapsulated by a `String` instance after its creation, sharing a reference to a `String` that forms the internal data encapsulated by an other object is harmless, as it will not be possible to change the object using the reference. This applies to any immutable object.

Objects are *immutable* if their class provides no way to change the internal state of the object after initialization. By extension, I call a class that yields immutable objects an *immutable class*.¹¹ Unfortunately, in Java and most other programming languages, there is no mechanism to guarantee that a class yields immutable objects. For the designer of a class, the only way to ensure immutability is to carefully design the class to prevent any modification (e.g., by providing no setter methods, leaking no reference, etc.). When relying on library classes (such as `String`), unless we are willing to personally inspect the source code of the class, we have to trust the documentation. Generally speaking, immutable objects have many advantages. In the context of this chapter, the immediate benefit is to support sharing information encapsulated in an object without breaking encapsulation. Chapter 4 provides additional insights that can help with the design of immutable classes. For now, it is sufficient to say that immutability is a desirable design property in many cases.

Let us conclude this introduction to immutability by defining class `Card` to be immutable. First, we rely on our two enumerated types `Rank` and `Suit` which we assume to be immutable.¹² With the following declaration, class `Card` will be immutable:

```
public class Card {
    private Rank aRank;
    private Suit aSuit;

    public Card(Rank pRank, Suit pSuit) {
        aRank = pRank;
        aSuit = pSuit;
    }

    public Rank getRank() {
        return aRank;
    }

    public Suit getSuit() {
        return aSuit;
    }
}
```

In this definition of the class, the only way to set the values of the two instance variables is through the constructor call which, by definition, is only executed once for each object. The fields are private, so they cannot be accessed from outside the

¹¹ This is a slight abuse of language because, technically speaking, it makes no sense for a class to be immutable. However, *immutable class* is a more convenient term than *class that yields immutable objects*.

¹² Simple enumerated types, which only enumerate values, are immutable. Although it is technically possible to define enumerated types that are *not* immutable, this is not a good idea. See Chapter 4.

class. There are only two methods. Although they are public, neither changes (or *mutates*) the state of the object. Finally, although the methods return a reference to the content of a field, the type of these fields is immutable, so it will not be possible to change the state of the referenced objects in any case. The class is thus immutable.

Code Exploration: JetUML · Rectangle

Creating objects derived from immutable objects.

In addition to its generated methods (e.g., `x()`, `y()`) and user-defined methods (e.g., `maxX()`, `maxY()`), the interface of this record type also includes methods that create *new* `Rectangle` instances as derivatives of the implicit parameter. For instance, `translated(int, int)` returns a new instance of `Rectangle` that is a translated version of the implicit parameter. The method is called `translated` instead of `translate`, because `translate` would imply the translation of the implicit parameter. This approach is necessary because, this record is intended to be immutable, so it is not possible to translate the implicit parameter. The pattern of “modifying” an immutable object by returning a modified version of the (unmodified) implicit parameter is common. The library class `String` provides many examples, such as `substring(int)`, which returns a new `String` instance that is a substring of the object on which the method is called.

2.7 Exposing Internal Data

In many cases the objects of the classes we define will need to expose part of the information they encapsulate to other objects. How can we do this without breaking encapsulation? As often in software design, there are different options, each with its strengths and weaknesses. For the sake of discussion, let us consider that we want to design our `Deck` class so that it is possible to find out what cards are in a deck.

```
public class Deck {  
    private List<Card> aCards = new ArrayList<>();  
}
```

As discussed above, adding a getter method that simply returns `aCards` is out of the question, as this allows code outside the class `Deck` to modify the internal representation of a `Deck` instance.

Extended Interface

One solution is to extend the interface of the class to include access methods that only return references to immutable objects. In our case, we could accomplish this goal by adding two methods to the `Deck` class:

```

public int size() {
    return aCards.size();
}

public Card cardAt(int pIndex) {
    return aCards.get(pIndex);
}

```

If class `Card` is immutable, this solution fulfills its mandate. However, it is somewhat inelegant if client code typically needs to access all the cards in the deck. In such a situation, the code would become cluttered with calls to `size()` and `for` loops going over all indexes. Code might also need to be written to check that the argument to `cardAt` is not out of bounds.

Returning Copies

Another option, which mimics returning a reference to the field `aCards` without breaking encapsulation, is to return a *copy* of the list stored in `aCards`. Thus, we could add a new method:

```

public List<Card> cards() {
    return new ArrayList<>(aCards);
}

```

This code relies on the behavior of the constructor `ArrayList(Collection)`, which creates a new `ArrayList` and initializes this list with all the elements in the collection, in the same order. Thus, a client would receive a reference to a different list of cards, with the same cards, as illustrated in Figure 2.6.

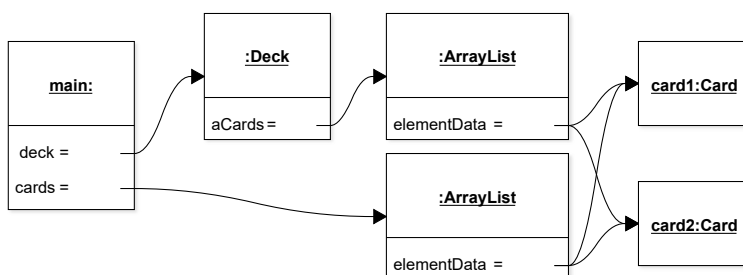


Fig. 2.6 Reference to a copy of a list of cards

Assuming `Card` is immutable, we have a valid solution to expose the content of a `Deck` to clients. Figure 2.6 shows the result of executing:

```

public static void main(String[] pArgs) {
    List<Card> cards = deck.cards();
}

```

We see that it is not possible to change the internal state of `Deck` from a reference to its cards. There are other strategies for returning a copy of a data structure or a wrapper for it. Ultimately, the details of the implementation do not matter as much as the central idea, which is to return a different object that has all the same information as the internal structure we wish to keep encapsulated.

Although it looks like a simple idea, copying objects is actually a tricky topic, because it requires deciding how deep to copy the object graph. So far, we assumed that `Card` objects were immutable, so it was sufficient to perform a *shallow copy*. A shallow copy of a list is a copy of the list with shared references to the elements in the original list (that is, the elements are not copied). But what if `Card` instances were mutable? In this case the above solution would not offer good encapsulation, because it would become possible to change the state of a deck without going through its interface, for example:

```
public static void main(String[] pArgs) {
    Deck deck = new Deck();
    deck.cards().get(0).setSuit(Suit.HEARTS);
}
```

With mutable `Card` instances, to implement the copying solution correctly, we need to go one step further and copy all cards when we copy the list of cards encapsulated within a `Deck` instance. In turn, this introduces a new requirement, namely, to find a clean way to copy card objects.

A common technique for copying objects is to use a *copy constructor*. A copy constructor takes as argument an object of the same class, and (usually) copies matching field values:

```
public Card(Card pCard) {
    aRank = pCard.aRank;
    aSuit = pCard.aSuit;
}
```

In fact, the code above, where we use `new ArrayList<>(aCards)`, is an example of a copy constructor for `ArrayList`.¹³ To perform a deep(er) copy of our list of cards now becomes slightly more involved:

```
public List<Card> cards() {
    ArrayList<Card> copy = new ArrayList<>();
    for (Card card : aCards) {
        copy.add(new Card(card));
    }
    return copy;
}
```

However, this extended solution ensures that encapsulation would be preserved even with mutable `Card` objects. Java provides other mechanisms that support copying objects, including its *cloning* mechanism (see Section 6.6), metaprogramming (see Section 5.4) and its *serialization* mechanism (not covered in the book).

¹³ This constructor for `ArrayList` has a formal parameter of type `Collection`, which is a supertype of `ArrayList`. Hence, it is more general than a strict copy constructor, which would have `ArrayList` as its formal parameter. However, these variants are closely related.

Other Strategies

Copying objects is only one of many strategies for exposing information internal to an object while maintaining encapsulation. The Java class library provides another option through the use of *unmodifiable view collections*. An unmodifiable view is an unmodifiable wrapper for an underlying collection of objects. For example, the library method `Collection.unmodifiableList(List)` returns an unmodifiable wrapper around a list. As an alternative to copying a list, we could do:

```
public List<Card> cards() {  
    return Collections.unmodifiableList(aCards);  
}
```

Other strategies will be covered later in the book. These include iterators (see Section 3.5) and streams (see Section 9.6).

Code Exploration: JetUML · Diagram

Using unmodifiable collections in practice.

A `Diagram` object holds a collection of root nodes and a collection of edges. Client code can access these collections through the methods `rootNodes()` and `edges()`. These two methods return the corresponding collection wrapped in an unmodifiable view using the library method `Collections.unmodifiableList`.

2.8 Input Validation

One of the benefits of encapsulation is to make it difficult or impossible for client code to corrupt the value of a variable. Following the principles and guidelines presented in this chapter helps us achieve this goal. Let us consider the following implementation of class `Card`.

```
public class Card {  
    private Rank aRank;  
    private Suit aSuit;  
  
    public Card(Rank pRank, Suit pSuit) {  
        aRank = pRank;  
        aSuit = pSuit;  
    }  
  
    public Rank rank() {  
        return aRank; }  
  
    public Suit suit() {  
        return aSuit; }  
}
```

The encapsulation provided by this class is very good, but there remains a crack in the shell it provides: it is possible to create a new card with a null reference:

```
Card card = new Card(null, Suit.CLUBS);
```

For most use cases where a representation of a playing card is required, this would be incorrect. At least, I am not aware of any card game that involves a “null of Clubs”. Section 4.4 provides an extended discussion of the issue of null references, but for now we focus on the general problem of avoiding the creation of an invalid instance of class `Card`. For this purpose, one strategy is to modify the code that provides our functionality of interest so that it checks that the input is valid, and reports an error otherwise. In Java we typically use exception handling for this purpose (see Section A.8 in the appendix):

```
/**
 * ...
 * @throws IllegalArgumentException if pRank or pSuit is null
 */
public Card(Rank pRank, Suit pSuit) {
    if (pRank == null || pSuit == null) {
        throw new IllegalArgumentException();
    }
    aRank = pRank;
    aSuit = pSuit;
}
```

With this code, any attempt to create a `Card` instance with a null reference for either of the two fields will result in an exception being thrown. When an exception is thrown, the execution of the constructor does not complete normally, and thus does not return the newly created object. For this reason, it is now impossible to obtain an invalid `Card` object by calling the class’s constructor.

An important consequence of this input validation is that now the null check becomes an integral part of the implementation of the constructor. Like any other kind of functionality, users of the code should be aware of how a method or constructor behaves in response to its input. For this reason, it is necessary to document this behavior carefully. In the example above, the information about the exception being raised is provided using Javadoc’s `@throws` tag.

It is important to remember that, in object-oriented programming, the object that is the target of a method call is also an input to the method. As such, this input may need to be validated as well. Let us consider a slightly more complete version of the `Deck` class where we have added an implementation of a `draw()` method, along with a method to check whether the deck is empty:

```

public class Deck {
    private List<Card> aCards = new ArrayList<>();

    public boolean isEmpty() {
        return aCards.isEmpty();
    }

    public Card draw() {
        return aCards.removeLast();
    }
}

```

Calling method `draw()` on an instance of `Deck` that contains no card will result in an exception being thrown by method `removeLast`, which will propagate out of method `draw()`, causing it to terminate abnormally. This situation, however, is very different from the case above. In our `Card` constructor, we *explicitly* designed our code to detect a null reference being passed as argument, and to throw an exception in response. The code comment reflects this design decision. In the case of `draw()`, the exception is raised because we misused a library method in our implementation by passing *it* an invalid input. The resulting exception is `NoSuchElementException`. One sloppy way to deal with the situation is to simply document the exception as follows:

```

/**
 * ...
 * @throws NoSuchElementException if isEmpty()
 */
public Card draw() {
    return aCards.removeLast();
}

```

This approach, however, has two major drawbacks: it abuses the exception handling mechanism, and it can violate the principle of information hiding. A good design principle for exception handling is that exceptions should only be used for unpredictable situations. However, this is not our situation, because we can always determine with complete certainty whether the list will be empty by calling `isEmpty()`.¹⁴ As for information hiding, the reason why propagating `NoSuchElementException` violates the principle is that propagated exceptions that reflect design decisions of internal structures may be puzzling to interpret in the context of client code. In our case, it is ambiguous what “element” is referred to by `NoSuchElement...`. Overlooking the documentation, one may erroneously think that the deck was not empty but somehow an error occurred when retrieving the card at the top. Although the implications are not dramatic, the encapsulation of class `Deck` can be improved by avoiding this information leak. A solution that avoids both problems is thus to implement an explicit check, similarly to how we have done with the `Card` constructor. In this case, because the illegal argument is the implicit argument (the object that is the target of the call), it is clearer to use

¹⁴ This discussion assumes a single-threaded execution context. Concurrent programming is outside the scope of this book.

`IllegalStateException`. Chapter 4 discusses the concept of object state in more detail.

```
/**
 * ...
 * @throws IllegalStateException if the deck is empty
 */
public Card draw() {
    if (isEmpty()) {
        throw new IllegalStateException();
    }
    return aCards.removeLast();
}
```

Input validation is one option for ensuring that we only construct valid objects and use them in valid ways. As usual, this design decision has both benefits and drawbacks. The main benefit, as we have seen, is that the class is robust: client code can no longer corrupt the internal values in an object. The consequence, however, is that we have shifted the responsibility of the client code from *input validation* to *error handling*. Presumably, if the client code is written so that it is possible to raise an exception, it should also catch this exception:

```
try {
    card = deck.draw();
} catch (IllegalStateException exception) {
    // Recover
}
```

Another important consequence of input validation is that now we have additional input validation code to test, document, and maintain within our classes. In some cases, this extra burden may not be justified. For example, if we only create new cards in one location in the code, where it is clear that no null values are used, then the error-handling machinery for protecting against the possibility of null inputs would be excessive. In the next section, I describe a systematic way to think about input validity.

Code Exploration: JetUML · Version

Input validation when it is really needed.

The `Version` record represents a specific JetUML release number, for example, 2.1. The method `parse(String)` of this class shows a good example of a case where input validation is typically necessary. Because the input value is read from a file, we do not know what to expect. For example, a buggy program may have written the file with an invalid version number. The implementation of `parse` thus ensures that the input is valid, and throws an `IllegalArgumentException` if it is not the case, this time supplying an error message to the constructor of the exception.

2.9 Design by Contract

In the previous section, I pointed out the need for input validation to ensure that client code does not misuse an object. However, input validation may not be necessary if the client code is written in a way that precludes erroneous values. For example, the following code creates all the cards in the Clubs suit:¹⁵

```
List<Card> clubs = new ArrayList<>();
for (Rank rank : Rank.values()) {
    clubs.add(new Card(rank, Suit.CLUBS));
}
```

With code like this, no null reference can ever be provided as argument to the `Card` constructor, and we could consider omitting input validation. Unfortunately, the fact that the responsibility for ensuring that valid values are used in a program can rest either on the implementation of a class or on its client code creates a source of ambiguity. Let us again consider the interface of the `Card` constructor:

```
public Card(Rank pRank, Suit pSuit)
```

Without additional information, the following interpretations are possible about the behavior of this constructor:

- It validates the input and throws an exception if it is null, but this fact is not documented;
- It validates the input, but does something else if it is null (for example, use a default value);
- It does *not* validate the input, expects the client code to *only* pass valid values, and breaks in some undefined way if it receives invalid arguments;
- It does *not* validate the input, and client code can create cards with null values as long as it does not use a card for which either the rank or suit is null.

Ambiguity of this nature can very easily destroy the quality of a design, render code incomprehensible, and upset developers using it. Observing that the problem comes from the ambiguity about what is or should be a legal value for the arguments of the `Card` constructor, one solution is to define method and constructor signatures so that the ambiguity is minimized or eliminated.

The idea of *design by contract* is to follow a principled approach to the specification of interfaces. Although, in practice, method signatures already specify much of what is needed in an interface, they also leave room for ambiguity, as was shown by the example above. Diligent programmers can help eliminate ambiguities by stating the precise range of allowed values in a method's documentation. This is certainly better than nothing. However, design by contract goes further and provides a formal framework for reasoning about complete interface information. There is a lot to say about design by contract, so to keep things tractable I only provide an overview of a simplified version of the approach.

¹⁵ Method `values()` is a static method available for all enumerated types. It returns an array that contains all the enumerated values for the type in declaration order.

The main idea of design by contract is for method signatures (and related documentation) to provide a sort of contract between the client (the caller method) and the server (the method being called). This contract takes the form of a set of *preconditions* and a set of *postconditions*. A precondition is a predicate that must be true when a method starts executing. The predicate typically involves the value of the method's arguments, including the state of the target object upon which the method is called. Similarly, postconditions are predicates that must be true when the execution of the method is completed.¹⁶ Given preconditions and postconditions, the contract is basically that the method can only be expected to conform to the postconditions if the caller conforms to the preconditions. If a client calls a method without respecting the preconditions, the behavior of the method is undefined. In practice, design by contract is a great way to force us to think about all possible ways to use a method.

In the sample applications (see Appendix C) I follow a lightweight version of design by contract where preconditions are specified using Java statements in the comments using the Javadoc `@pre` tag and postconditions are specified using the tag `@post`.

```
/**
 * @pre pRank != null && pSuit != null
 */
public Card(Rank pRank, Suit pSuit) {
    // ...
}
```

It is possible to make pre- and postconditions (and any other predicates) *checkable* in Java using the `assert` statement:

```
public Card(Rank pRank, Suit pSuit) {
    assert pRank != null && pSuit != null;
    aRank = pRank;
    aSuit = pSuit;
}
```

The `assert` statement evaluates its predicate expression and raises an `AssertionError` if the result is false.¹⁷

Correctly implemented, design by contract helps prevent the tedious idiom of *defensive programming* where corner cases (such as null references) are checked for everywhere in the code. Additionally, the technique supports clear *blame assignment* while debugging: If a precondition check fails, the client (caller method) is to blame. If a postcondition check fails, the method being called is to blame. More generally, `assert` statements are a simple yet powerful tool to increase code quality and they can be used anywhere, not just for pre- and postconditions. Whenever an

¹⁶ The complete approach also involves the concept of *invariants*. In theory, invariants are predicates that are expected to remain true at all times. In the practice of design by contract, it is sufficient for invariants to be true at method entry and exit points.

¹⁷ Assertion checking is disabled by default in Java, so to use this properly it is necessary to add `-ea` (enable assertions) as a VM parameter when running Java.

assertion check fails, we know exactly where the problem is, and we can thus save on debugging time.

A final note about design by contract is that the addition of preconditions to a method's interface actually relieves us of the requirement to handle the condition. Hence, the code below is not properly designed because it *both* states that null references are not a valid input *and* handles them in a consistent way (by raising an exception). If a method checks for a certain type of input (like null references) and produces a well-defined behavior as the result, then this is part of the method's interface specification. When designing method interfaces, it is important to decide whether the method will be in charge of rejecting illegal values, or whether these will be specified as invalid. These are two different design choices. In the same vein, it must be emphasized that the `assert` statement is not a compact way to implement input validation. `AssertionErrors` are not meant to represent the presence of invalid values in a running program. Rather, they point to a design or implementation flaw in the code.

```
/**
 * @pre pRank != null && pSuit != null
 */
public Card(Rank pRank, Suit pSuit) {
    if (pRank == null || pSuit == null) {
        throw new IllegalArgumentException();
    }
    // ...
}
```

Code Exploration: JetUML · Rectangle

Design by contract in practice.

The methods of record `Rectangle` provide examples of different types of preconditions. The methods that take a reference type as argument (for example, `contains(Point)` and `contains(Rectangle)`) require that this argument be non-null. Other design decisions were possible. For example, it would have been possible to accept `null` as an argument to `contains`, and return `false` when this argument is provided. A downside of this alternative is that it makes the interface ambiguous: if `false` is returned, is it because the point was not contained in the rectangle, or because the point was not actually a point, but `null`? Section 4.4 provides additional reasons why null values are best avoided.

Code Exploration: Solitaire · Deck

Preconditions on the state of the target object.

The interface to method `draw` illustrates how preconditions can be a function of the state of the object. The interface of class `Deck` requires that the deck not be empty before `draw` is called. Here, this precondition can be expressed in terms of another method, `isEmpty()` (by negating the return value).

Insights

This chapter focused on how to follow the principles of encapsulation and information hiding when defining classes.

- Use classes to define how domain concepts are represented in code, as opposed to encoding instances of these concepts as values of primitive types (an antipattern called `PRIMITIVE OBSESSION†`);
- Use enumerated types to represent a value in a collection of a small number of elements that can be enumerated;
- Hide the internal implementation of an abstraction behind an interface that tightly controls how an abstraction can be used. Declare fields of a class `private`, unless you have a strong reason not to. Similarly, declare any method `private` if it should not be explicitly part of the type's interface;
- Ensure that the design of your classes prevents any code from modifying the data stored in an object of the class without using a method of the class. In particular, be careful to avoid leaking references to private fields of the class that refer to mutable objects;
- To provide information about the internal data in an object without violating encapsulation, strategies include extending the interface of the class, returning copies of internal objects, or using unmodifiable views;
- Object diagrams can help explain or clarify the structure of complex object graphs, or how references are shared;
- Make classes immutable if possible. In Java, it is only possible to ensure that a class is immutable through careful design and inspection;
- Input validation can be used to ensure that the objects of a class are created and used properly. However, this extra code comes at a cost as it needs to be documented, tested, and maintained.
- Use design by contract to avoid ambiguity in method signatures, and thereby help prevent the possibility that client code will misuse an instance of a class.

Further Reading

Chapter 6 of the book *Software Architecture: A Comprehensive Framework and Guide for Practitioners* by Vogel et al. [17] provides a well-organized overview of the different principles of software design, and how they are related. For a more historical perspective, the seminal paper on the principle of information hiding is Parnas's 1972 *On the Criteria to be Used in Decomposing Systems into Modules* [12]. The article contrasts two designs for a text-processing system, and argues for the superiority of the design that realizes information hiding over a sequential processing decomposition.

The section titled *Enum Types* in the Java Tutorial [11] provides additional insights on how enumerated types work. In Item 50 of the book *Effective Java* [2], Bloch discusses the creation of defensive copies of internal objects encapsulated by a class.

The article *Applying Design by Contract*, by its inventor Bertrand Meyer, provides an accessible overview of the technique [9].